



Introducing Today's Project!

In this project, I will demonstrate automated testing with GitHub Actions. I'm doing this project to learn how to set up CI/CD workflows, automate software testing, and improve code reliability through continuous integration practices. This project will also help me gain hands-on experience with GitHub Actions, workflow automation, and modern DevOps development processes.

Key services and concepts

Services I used were GitHub, GitHub Actions, Ollama, and ChromaDB, and key concepts I learned include version control, CI/CD pipelines, semantic testing, mock LLM integration, Retrieval-Augmented Generation (RAG), vector databases, and local LLM workflows.

Challenges and wins

This project took me approximately several hours to complete, the most challenging part was resolving dependency and CI testing issues while ensuring deterministic outputs, and it was most rewarding to successfully build and automate a working RAG pipeline with Ollama and ChromaDB.

Why I did this project

I did this project because I wanted to learn about DevOps, CI/CD automation, semantic testing, and Retrieval-Augmented Generation (RAG) systems, and one thing I'll apply from this is using automated workflows and deterministic testing to improve the reliability and deployment of AI applications.



Setting Up Your RAG API

I'm setting up my RAG API by connecting a vector database to a language model using FastAPI. A RAG API retrieves information by finding the most relevant documents in your knowledge base and using them to generate accurate answers. This foundation is needed for CI/CD because the system has multiple parts that can break, so automating tests ensures everything works before changes go live.

Local API verification

I tested my RAG API by running the `curl -X POST "http://127.0.0.1:8000/query" -G --data-urlencode "q=What is Kubernetes?"` command and the API responded with the answer as we can see in the screenshot. This confirms that the RAG APP is working perfectly.

```
((venv) )
Abdullahi Yakubu@DESKTOP-08HCGPJ MINGW64 ~/Desktop/nextwork-rag-api
$ curl -X POST "http://127.0.0.1:8000/query" -G --data-urlencode "q=What is Kubernetes?"
{"answer": "Sure, I'd be happy to provide you with the answer! Kubernetes is a highly popular container orchestration platform used to manage containers at scale. It allows developers to deploy, manage, and scale applications within their Kubernetes clusters efficiently and effectively. Kubernetes is a software component of Kubernetes that enables automated deployment and management of Kubernetes resources such as pods, namespaces, and services."}((venv) )
Abdullahi Yakubu@DESKTOP-08HCGPJ MINGW64 ~/Desktop/nextwork-rag-api
$
```



Initializing Git and Pushing to GitHub

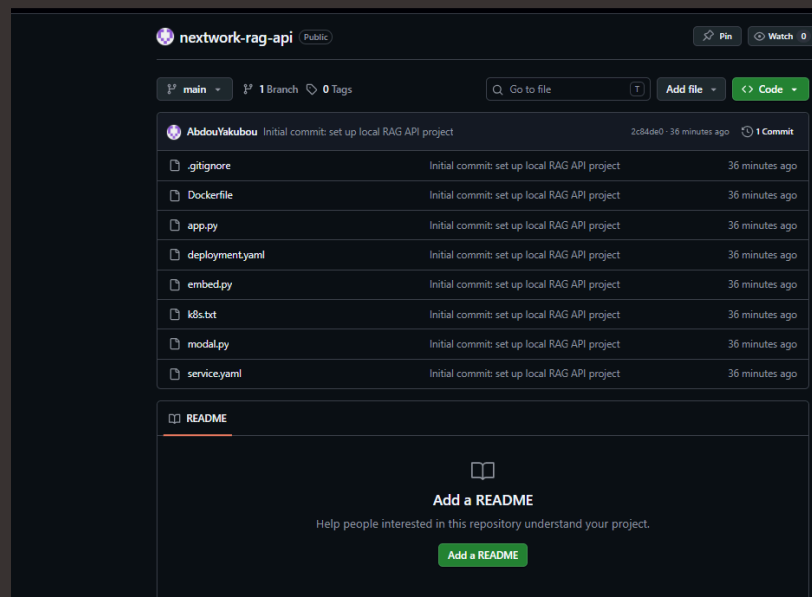
I'm initializing Git by creating a repository, Git tracks changes through commits and file snapshots, and version control enables CI/CD to automate testing and deployment whenever code is updated.

Git initialization and first commit

I initialized Git by creating a repository with `git init`, then I staged and committed my files using `git add` and `git commit`, and the `.gitignore` file helps by excluding unnecessary or sensitive files from being tracked.

Pushing to GitHub for CI/CD

Pushing to GitHub means uploading local commits to a remote repository, and this enables CI/CD because automated workflows can test, build, and deploy the code whenever updates are pushed.





Creating Semantic Tests

I'm creating semantic tests that verify the meaning and relevance of responses, unlike unit tests that checks code, semantic tests validate output quality and accuracy, and these tests ensure quality by confirming taht the system behaves correctly from a user perspective.

Non-deterministic output observation

When I ran the query multiple times, I noticed the responses were inconsistent, this is a problem because unreliable outputs can break automated validation, and for CI/CD to work reliably, we need deterministic and repeatable results.



Adding Mock LLM Mode

I'm adding mock LLM mode to stimulate consistent responses from the model , this solves the non-determinism problem by returning same outputs for every test run, and reliable testing requires stable, repeatable, and verifiable results.

Mock LLM mode for CI testing

Mock LLM mode returns the retrieved text directly, which makes tests deterministic and consistent, without mock mode tests would produce unpredictable results due to varying LLM outputs, and for automated CI we need stable and repeatable test behavior.



Creating GitHub Actions Workflow

I'm creating a GitHub Actions workflow file that defines automated CI steps, the workflow automates testing by running checks whenever code changes are pushed, and when I push code it will automatically execute the testing pipeline.

Workflow automation and CI testing

I created the workflow file in the `.github/workflows` directory, I pushed it using Git commands like `git add`, `git commit`, and `git push`, and once on GitHub the workflow will automatically run the defined CI pipeline.



Testing Data Quality

I'm triggering the CI workflow by pushing new code to GitHub, the workflow will test the application and its automated checks, and I expect it to fail because the project still contains a known issue introduced for testing the pipeline.

Data quality and CI protection

The missing keyword was orchestration and was identified by the semantic identification check, the semantic test failed because the response has a missing keyword as identified in the mock. and without CI this degraded content would have been merged into production unnoticed.

The screenshot displays a GitHub Actions workflow run for the 'test-and-build' job. The workflow is shown in a dark-themed interface. On the left, a sidebar contains links for 'Summary', 'test-and-build', 'Run details', 'Usage', and 'Workflow file'. The main area shows the 'test-and-build' job, which failed 2 minutes ago. The job steps are listed on the right, with 'Run semantic tests' highlighted in red. The 'Run semantic tests' step is expanded, showing the following code:

```
1 # Run python semantic_test.py
2 python semantic_test.py
3 shell: /usr/bin/bash -e {0}
4
5 pythonLocation: /opt/hostedtoolcache/Python/3.11.35/x64
6 PIP_COMPILE_PATH: /opt/hostedtoolcache/Python/3.11.35/x64/lib/pipxconfig
7 Python_ROOT_DIR: /opt/hostedtoolcache/Python/3.11.35/x64
8 Python_ROOT_DIR: /opt/hostedtoolcache/Python/3.11.35/x64
9 Python_ROOT_DIR: /opt/hostedtoolcache/Python/3.11.35/x64
10 cd /tmp/2024/ /opt/hostedtoolcache/Python/3.11.35/x64/lib
11 Traceback (most recent call last):
12   File "/home/runner/work/network-rag-api/network-rag-api/semantic_test.py", line 18, in <module>
13     test_kubernetes_query()
14   File "/home/runner/work/network-rag-api/network-rag-api/semantic_test.py", line 12, in test_kubernetes_query
15     assert "orchestration" in answer.lower(), "Missing 'orchestration' keyword"
16     ~~~~~^~~~~~
17 AssertionError: Missing 'orchestration' keyword
18 Error: Process completed with exit code 1.
```

The error message at the bottom of the expanded step is 'AssertionError: Missing 'orchestration' keyword'. The job status is 'Failed' and the overall workflow status is 'Failed'.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

